

Backend API Architectural Blueprint & Integration Analysis

Target Architecture: ASP.NET Core Web API (.NET 8/9) with SQL Server, EF Core, and React Client (TypeScript)

This document contains a comprehensive analysis of the frontend codebase of the School Management System (SMS) and outlines the backend REST API architecture, database mappings, execution roadmaps, and verification plans.

1. Global System Flow & Navigation Lifecycle

The system utilizes Role-Based Access Control (RBAC) to govern access to different portals. The lifecycle of a user session involves the following sequences:

```
graph TD
  A[Login Screen] -->|POST /api/auth/login| B{MFA Required?}
  B -->|Yes| C[OTP Request]
  C -->|POST /api/auth/otp/send| D[User Receives Email/SMS]
  D -->|POST /api/auth/otp/verify| E[Auth Success: JWT Token Issued]
  B -->|No| E
  E -->|Fetch Session Headers| F[App Mounts Sidebar & Top Header]
  F -->|GET /api/academic-years| G[Load Active Academic Year]
  F -->|GET /api/branches/authorized| H[Load Authorized Branches]
  F -->|GET /api/users/profile| I[Fetch User Profile Details]
  I -->|RBAC Dashboard Routing| J[Module Navigation]
```

Screen Navigation & API Traversal Flow

- **Login & Session Hydration:**
 - API Calls: `POST /api/auth/login`, `GET /api/academic-years`, `GET /api/branches/authorized`, `GET /api/users/profile`.
 - Headers Injected (via `client.ts`): `Authorization: Bearer `, `X-Branch-Id: `, `X-Academic-Year-Id: `.
- **Student Directory Dashboard:**
 - Action: Teacher/Admin clicks "Student Management" in sidebar.
 - API Calls:
 - `GET /api/classes` (Populate class select dropdown)
 - `GET /api/students?class=§ion;=&search;=&page;=1` (Populate table)
 - Modal Actions:
 - *Create Student*: `GET /api/transport/routes` and `GET /api/hostels/blocks` to populate facility dropdowns. Click Save -> `POST /api/students`.
 - *Promote Student*: Click Promote -> `POST /api/students/promote` (updates class/section mapping and logs history).
 - *Parent Messaging*: Click message button -> `POST /api/communication/dispatch-parent-sms`.
- **Student Attendance Cockpit:**
 - Action: Teacher opens Attendance sheet.
 - API Calls:
 - `GET /api/classes` (Fetch classes dropdown)
 - `GET /api/subjects/dropdown` (Filter by assigned subjects)
 - `GET /api/attendance/students?class=§ion;=&subject;=&date;=.` (Fetch attendance checklist roster).
 - Save Attendance: `POST /api/attendance` (Bulk saves Present, Absent, Late, or Half-day logs).
- **Marks Entry Register:**

- Action: Evaluator enters grades.
- API Calls:
 - `GET /api/classes` and `GET /api/subjects` (Configure context filters).
 - `GET /api/exams/setup?class=` (Load active exam contexts: FA1, FA2, SA1, SA2).
 - `GET /api/exams/marks?examId=&subject;=` (Fetch students marks list).
 - Save Draft: `PUT /api/exams/marks/draft` (Saves scores dynamically).
 - Publish: `POST /api/exams/marks/publish` (Locks rows, sends notifications).
- **Homework Distribution:**
 - Action: Click Homework Module.
 - API Calls:
 - `GET /api/homework?status=` (Load full homework grid).
 - Create Homework: Toggle selected students -> `GET /api/students/dropdown?class=§ion;=` (Hydrate student checkbox roster).
 - Publish: `POST /api/homework` (Payload contains `publishToType` ('Class' | 'Students') and `publishedStudentIds` array).

2. Reusable Endpoint Designs

Rather than duplicating logic across modules, the following core APIs serve as system-wide, shared endpoints:

Endpoints	HTTP Method	Reuse Contexts	Description
`/api/lookups/classes`	`GET`	Student list, Attendance, Marks, Timetable, Fees, Transport, Hostel	Fetch available classes & sections.
`/api/lookups/subjects`	`GET`	Timetable, Marks Entry, Homework, Staff Profile	Fetch subject list filtered by class/department.
`/api/lookups/students`	`GET`	Homework selectors, Hostel assignments, Transport assignments	Roster checklist selector with search.
`/api/files/upload`	`POST`	Admission forms, Homework attachments, Staff documents, Event attachments	Unified multipart image/document upload handler.
`/api/lookups/academic-years`	`GET`	Header Selector, Exam Configs, Promotion History, Fees Setup	Fetch school academic cycles.

3. Module-Wise Backend API Catalogue

Module A: Authentication & Core Setup (Auth)

1. Login Authentication

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/auth/login`
- **Purpose:** User login verification and JWT issuance.
- **Request DTO** (`LoginRequestDto`):

```
{ "emailOrPhone": "admin@stxaviers.edu", "password": "SecurePassword123" }
```

- **Response DTO** (`LoginResponseDto`):

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "id": "USR-001", "name": "Dr. Eleanor Vance", "email": "admin@stxaviers.edu", "role": "Admin", "avatar": "https://images.unsplash.com/...jpg" }, "roles": ["Admin", "Principal"] }
```

- **Validation Rules:**
 - `emailOrPhone` cannot be empty.
 - `password` must meet complexity: min 8 chars, 1 uppercase, 1 special character.
- **Auth Requirements:** Anonymous access.
- **Related Tables:** `AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`.
- **Dependencies:** None.

2. OTP Code Request

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/auth/otp/send`
- **Request DTO** (`OtpRequestDto`):

```
{ "emailOrPhone": "admin@stxaviers.edu", "deliveryMethod": "Email" }
```

- **Response DTO** (`GenericSuccessDto`):

```
{ "success": true, "message": "OTP code dispatched successfully." }
```

- **Validation Rules:** `emailOrPhone` must be a valid email or phone formatting.
- **Auth Requirements:** Anonymous access.
- **Related Tables:** `UserOtpCodes`.
- **Dependencies:** User account must exist.

3. Verify OTP Code

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/auth/otp/verify`
- **Request DTO** (`OtpVerifyDto`):

```
{ "emailOrPhone": "admin@stxaviers.edu", "otpCode": "123456" }
```

- **Response DTO** (`OtpVerifyResponseDto`):

```
{ "verified": true, "otpVerificationToken": "otp-verification-guid-token-XYZ" }
```

- **Validation Rules:** `otpCode` must be exactly 6 digits.
- **Auth/Role Requirements:** Anonymous access.
- **Related Tables:** `UserOtpCodes`.

- **Dependencies:** OTP must have been sent.

4. Reset Password with OTP

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/auth/otp/reset-password`
- **Request DTO (`ResetPasswordWithOtpDto`):**

```
{ "emailOrPhone": "admin@stxaviers.edu", "otpCode": "123456", "newPassword": "NewSecurePassword123!", "confirmPassword": "NewSecurePassword123!" }
```

- **Response DTO (`GenericSuccessDto`):**

```
{ "success": true, "message": "Password changed successfully." }
```

- **Validation/Constraint Rules:** `newPassword` must match `confirmPassword`.
- **Auth Requirements:** Anonymous access.
- **Related Tables:** `AspNetUsers`.
- **Dependencies:** OTP must be validated.

Module B: Academics Setup

1. Fetch Class Roster Configuration

- **HTTP Method:** `GET`
- **Endpoint URL:** `/api/classes`
- **Purpose:** Fetch details of classes, sections, assigned teachers, and subjects.
- **Response DTO (`List`):**

```
[ { "id": "CL-9", "name": "Class 9", "sections": ["A", "B"], "sectionTeachers": { "A": "Sarah Jenkins", "B": "Jonathan Miller" }, "subjects": ["Mathematics", "Physics", "Chemistry"] } ]
```

- **Auth/Role Requirements:** JWT Authentication. Roles: `Super Admin`, `Admin`, `Teacher`, `Principal`.
- **Related Tables:** `Classes`, `Sections`, `ClassSectionTeacherAssignments`, `ClassSubjectMappings`.
- **Dependencies:** None.

2. Create Academic Class

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/classes`
- **Request DTO (`CreateClassDto`):**

```
{ "name": "Class 13", "sections": ["A", "B"], "sectionTeachers": { "A": "STF-01", "B": "STF-02" }, "subjects": ["Advanced Calculus", "Economics"] }
```

- **Response DTO (`ClassConfigDto`):** Returns the created class.
- **Validation Rules:** `name` must be unique in active academic year.
- **Auth/Role Requirements:** `Super Admin`, `Admin`.
- **Related Tables:** `Classes`, `Sections`, `ClassSectionTeacherAssignments`, `ClassSubjectMappings`.
- **Dependencies:** Academic Year Master record.

3. Update Academic Class

- **HTTP Method:** `PUT`
- **Endpoint URL:** `/api/classes/{id}`

- **Request DTO (`UpdateClassDto`)**: Similar to `CreateClassDto`.
- **Auth/Role Requirements**: `Super Admin`, `Admin`.

4. Delete Academic Class

- **HTTP Method**: `DELETE`
- **Endpoint URL**: `/api/classes/{id}`
- **Auth/Role Requirements**: `Super Admin`, `Admin`.
- **Database Action**: Restrict deletion if students or schedules are linked (Cascading protection).

5. Fetch Subjects Directory

- **HTTP Method**: `GET`
- **Endpoint URL**: `/api/subjects`
- **Purpose**: Load subject catalog for curriculum mapping.
- **Response DTO (`List`)**:

```
[ { "id": "SUB-101", "subjectId": "MAT-01", "name": "Mathematics", "code": "M101", "departmentId": "DEP-01", "departmentName": "Science Department", "isPractical": false, "labRequired": false } ]
```

- **Auth/Role Requirements**: JWT Authentication. Roles: `Admin`, `Teacher`, `Principal`.
- **Related Tables**: `Subjects`, `Departments`.
- **Dependencies**: Departments database must exist.

6. Create Subject

- **HTTP Method**: `POST`
- **Endpoint URL**: `/api/subjects`
- **Request DTO (`CreateSubjectDto`)**:

```
{ "name": "Quantum Physics", "code": "PHY-401", "departmentId": "DEP-01", "isPractical": true, "labRequired": true }
```

- **Response DTO (`SubjectDto`)**.
- **Validation Rules**: `code` must be unique.
- **Auth/Role Requirements**: `Super Admin`, `Admin`.

Module C: Student Management

1. Student Roster Directory (Search, Filter, Page)

- **HTTP Method**: `GET`
- **Endpoint URL**: `/api/students`
- **Purpose**: Fetch filtered, paginated student listings for Student Management Roster.
- **Query Parameters**: `class` (optional), `section` (optional), `search` (optional), `page` (default 1), `limit` (default 10).
- **Response DTO (`PagedResultDto`)**:

```
{ "totalCount": 120, "items": [ { "id": "STD-001", "admissionNo": "ADM-2026-009", "rollNo": "001", "firstName": "Rahul", "lastName": "Sharma", "gender": "Male", "dob": "15/08/2012", "className": "Class 10", "section": "A", "fatherName": "Manish Sharma", "fatherPhone": "9876543210", "email": "rahul.sharma@gmail.com", "status": "Active", "avatar": "https://...", "totalFee": 45000, "paidFee": 30000, "dueFee": 15000, "attendancePct": 92.5, "gpa": 8.5 } ] }
```

- **Auth/Role Requirements**: JWT Authentication. Roles: `Admin`, `Teacher`, `Principal`, `HR`, `Accountant`.

- **Related Tables:** `Students`, `FeeLedger`, `AttendanceLogs`.
- **Dependencies:** Classes, Academic Year.

2. Create Student (Admission Enrollment)

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/students`
- **Request DTO (`CreateStudentDto`):**

Matches all fields inside the frontend `StudentFormModal.tsx` form. Includes:

- **Personal:** `firstName`, `lastName`, `gender`, `dob`, `bloodGroup`, `religion`, `casteCategory`, `className`, `section`, `branch`, `studentType`.
- **Parents:** `fatherName`, `fatherPhone`, `fatherOccupation`, `fatherEmail`, `motherName`, `motherPhone`.
- **Facility Options:** `transportRequired`, `routeId`, `pickupPointId`, `hostelBlock`, `hostelRoom`, `hostelBed`.
- **Response DTO (`StudentDto`).**
- **Validation Rules:**
 - `dob` must indicate age between 3 and 20.
 - `fatherPhone`, `motherPhone` must be 10 digits.
 - `admissionNo` auto-generated on backend.
- **Auth/Role Requirements:** `Admin`, `Registrar`.
- **Related Tables:** `Students`, `StudentTransportAssignment`, `StudentHostelAssignment`, `FeeLedger`.

3. Update Student

- **HTTP Method:** `PUT`
- **Endpoint URL:** `/api/students/{id}`
- **Request DTO (`UpdateStudentDto`):** Contains fields to update.
- **Auth/Role Requirements:** `Admin`.

4. Promote Student

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/students/promote`
- **Request DTO (`PromoteStudentDto`):**

```
{ "studentIds": ["STD-001", "STD-002"], "targetClass": "Class 11", "targetSection": "A", "targetBranch": "Main Campus", "targetAcademicYear": "2027-2028" }
```

- **Response DTO (`GenericSuccessDto`).**
- **Auth/Role Requirements:** `Super Admin`, `Admin`.
- **Related Tables:** `Students`, `PromotionHistory`.

5. Transfer Student (Exit Clearance)

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/students/{id}/transfer`
- **Request DTO (`TransferStudentDto`):**

```
{ "relievingDate": "2026-08-01", "reason": "Family Relocation", "clearanceStatus": "Cleared" }
```

- **Response DTO (`GenericSuccessDto`).**
- **Auth/Role Requirements:** `Admin`.

- **Related Tables:** `Students`, `StudentExits`.
- **Dependencies:** Checks if `dueFee` in `FeeLedger` is 0 (Financial block).

Module D: Staff & User Management

1. Staff Directory (Search, Filter, Page)

- **HTTP Method:** `GET`
- **Endpoint URL:** `/api/staff`
- **Query Parameters:** `category` ('Teacher' | 'Staff'), `department` (optional), `search` (optional), `page`, `limit`.
- **Response DTO (`PagedResultDto`):**

```
{ "totalCount": 45, "items": [ { "id": "STF-01", "empId": "EMP-001", "firstName": "Jonathan", "lastName": "Miller", "designation": "Senior Lecturer", "department": "Mathematics", "employeeCategory": "Teacher", "email": "jonathan.miller@school.edu", "phone": "9876543001", "status": "Active" } ] }
```

- **Auth/Role Requirements:** `Super Admin`, `Admin`, `HR`, `Principal`.
- **Related Tables:** `StaffProfiles`.

2. Create Staff Profile

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/staff`
- **Request DTO (`CreateStaffDto`):**

Detailed payload matching `StaffFormModal.tsx` including:

- **Personal:** `firstName`, `lastName`, `designation`, `department`, `employeeCategory`, `email`, `phone`, `gender`, `dob`, `joiningDate`.
- **Banking:** `bankDetails` object (`accountNumber`, `bankName`, `ifscCode`).
- **Response DTO (`StaffDto`).**
- **Validation Rules:** `email` must be unique, `ifscCode` must validate against Reserve Bank schema pattern.
- **Auth/Role Requirements:** `Admin`, `HR`.
- **Related Tables:** `StaffProfiles`, `StaffBankDetails`, `AspNetUsers`.

3. Upload Staff Onboarding Document

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/staff/{id}/documents`
- **Purpose:** Upload credential scans (Aadhaar, B.Ed certificates, Offer letters).
- **Request Format:** Multipart Form Data. File bytes + payload parameters.
- **Response DTO (`StaffDocumentDto`):**

```
{ "id": "DOC-99", "title": "Aadhaar Scan", "type": "Aadhaar Card", "fileUrl": "/uploads/documents/EMP-001_aadhaar.pdf", "uploadedDate": "2026-08-03", "verificationStatus": "Pending Verification" }
```

- **Auth/Role Requirements:** `Admin`, `HR`, or User ID equals `id` (Self Service Upload).
- **Related Tables:** `StaffDocuments`.
- **Dependencies:** Staff record must exist.

4. Verify Onboarding Document

- HTTP Method: `PATCH`
- Endpoint URL: `/api/staff/{staffId}/documents/{docId}/verify`
- Request DTO (`VerifyDocRequestDto`):

```
{ "status": "Verified", "remarks": "Document details match the system entry" }
```

- Response DTO (`StaffDocumentDto`).
- Auth/Role Requirements: `HR`, `Principal`, `Admin`.
- Related Tables: `StaffDocuments`.

Module E: Attendance Cockpit

1. Fetch Student Roster for Marking Attendance

- HTTP Method: `GET`
- Endpoint URL: `/api/attendance/students`
- Query Parameters: `class`, `section`, `subject` (optional), `date` (format: YYYY-MM-DD), `period` (optional).
- Response DTO (`List`):

```
[ { "studentId": "STD-001", "rollNo": "001", "studentName": "Rahul Sharma", "status": "Present", "remarks": "On-time arrival" } ]
```

- Auth/Role Requirements: JWT Authentication. Roles: `Teacher`, `Admin`, `Principal`.
- Related Tables: `Students`, `DailyAttendance`.

2. Submit Bulk Attendance Sheet

- HTTP Method: `POST`
- Endpoint URL: `/api/attendance`
- Purpose: Register bulk attendance (Present, Absent, Late, HalfDay).
- Request DTO (`BulkAttendanceSubmitDto`):

```
{ "class": "Class 10", "section": "A", "subject": "Mathematics", "date": "2026-08-03", "period": "Period 1", "records": [ { "studentId": "STD-001", "status": "HalfDay", "remarks": "Left school early with parent slip" }, { "studentId": "STD-002", "status": "Present", "remarks": "" } ] }
```

- Response DTO (`GenericSuccessDto`).
- Validation Rules:
 - `date` cannot be in the future.
 - `status` must be within: `Present`, `Absent`, `Late`, `HalfDay`.
- Auth/Role Requirements: `Teacher`, `Admin`.
- Related Tables: `DailyAttendance`.
- Dependencies: Classes, Subjects.

Module F: Homework & Assignments

1. Create Homework

- HTTP Method: `POST`

- **Endpoint URL:** `/api/homework`

- **Request DTO (`CreateHomeworkDto`):**

```
{ "title": "Algebra Worksheet 2", "className": "Class 10", "section": "A", "subject": "Mathematics",  
"dueDate": "2026-08-05", "description": "Solve Equations 1 to 10 on page 45.", "publishToType": "Students",  
"publishedStudentIds": ["STD-001", "STD-002"], "status": "Published" }
```

- **Response DTO (`HomeworkDto`).**

- **Validation Rules:**

- `dueDate` must be a future date relative to server.
- If `publishToType` is `Students`, `publishedStudentIds` cannot be empty.

- **Auth/Role Requirements:** `Teacher`, `Admin`.

- **Related Tables:** `Homeworks`, `HomeworkTargets`.

2. Fetch Homework Listing (Student View)

- **HTTP Method:** `GET`

- **Endpoint URL:** `/api/homework/student/{studentId}`

- **Purpose:** Load published homework relevant to a student (either published to the whole class or targeted to this student).

- **Response DTO (`List`):**

```
[ { "id": "HW-401", "title": "Algebra Worksheet 2", "subject": "Mathematics", "dueDate": "2026-08-05",  
"description": "Solve Equations 1 to 10 on page 45.", "status": "Published", "teacherName": "Jonathan Miller"  
} ]
```

- **Auth/Role Requirements:** JWT Authentication. Roles: `Student`, `Parent`, `Teacher`, `Admin`.

- **Related Tables:** `Homeworks`, `HomeworkTargets`.

Module G: Fees & Financials

1. Setup Class Fee Structure

- **HTTP Method:** `POST`

- **Endpoint URL:** `/api/finance/fee-structures`

- **Request DTO (`CreateFeeStructureDto`):**

```
{ "academicYear": "2026-2027", "branch": "Main Campus", "className": "Class 10", "studentCategory":  
"Regular", "items": [ { "feeHeadId": "FHD-01", "amount": 25000 }, { "feeHeadId": "FHD-02", "amount": 5000 } ]  
}
```

- **Response DTO (`FeeStructureResponseDto`).**

- **Auth/Role Requirements:** `Super Admin`, `Admin`, `Accountant`.

- **Related Tables:** `FeeStructures`, `FeeStructureItems`.

- **Dependencies:** Fee Heads Master.

2. Collect Fee Payment

- **HTTP Method:** `POST`

- **Endpoint URL:** `/api/finance/payments`

- **Purpose:** Record payment, generate a receipt number, calculate fine, and deduct scholarships/discounts.

- **Request DTO (`FeePaymentRequestDto`):**

```
{ "studentId": "STD-001", "amountPaid": 20000, "paymentMode": "Online", "transactionId": "TXN987654321",  
"paymentDate": "2026-08-03", "remarks": "Q1 fee paid online" }
```

• **Response DTO** (`FeePaymentResponseDto`):

```
{ "receiptNo": "REC-2026-0941", "studentName": "Rahul Sharma", "amountPaid": 20000, "dueBalance": 5000, "status": "Partial" }
```

- **Validation Rules:**
 - `amountPaid` must be greater than 0.
 - `paymentMode` must be: `Cash`, `Card`, `Online`, `Cheque`.
- **Auth/Role Requirements:** `Accountant`, `Admin`.
- **Related Tables:** `FeePayments`, `StudentFeeLedgers`, `FinanceTransactions`.
- **Dependencies:** Student Fee Ledger mapping.

Module H: Exams & Grading System

1. Submit Student Exam Marks

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/exams/marks`
- **Request DTO** (`SubmitMarksRequestDto`):

```
{ "examId": "EXM-01", "subject": "Mathematics", "maxMarks": 100, "passMarks": 35, "records": [ { "studentId": "STD-001", "marksObtained": 85, "isAbsent": false, "remarks": "Excellent work" }, { "studentId": "STD-002", "marksObtained": 0, "isAbsent": true, "remarks": "Medical leave absent" } ] }
```

- **Response DTO** (`GenericSuccessDto`).
- **Validation Rules:**
 - `marksObtained` cannot exceed `maxMarks`.
 - `marksObtained` must be ≥ 0 .
- **Auth/Role Requirements:** `Teacher`, `Admin`.
- **Related Tables:** `ExamMarks`.
- **Dependencies:** Exam Setup.

2. Revalue Exam Score

- **HTTP Method:** `PUT`
- **Endpoint URL:** `/api/exams/marks/{id}/revalue`
- **Request DTO** (`RevalueMarksRequestDto`):

```
{ "newMarks": 88, "reason": "Re-addition calculation error", "type": "Revaluation" }
```

- **Response DTO** (`GenericSuccessDto`).
- **Auth/Role Requirements:** `Admin`, `Principal`.
- **Related Tables:** `ExamMarks`, `ExamMarkAuditHistory`.
- **Dependencies:** Original score must be locked/published.

Module I: Hostel Management

1. Allocate Room Bed to Student

- **HTTP Method:** `POST`

- **Endpoint URL:** `/api/hostels/allocations`
- **Request DTO** (`HostelAllocationRequestDto`):

```
{ "studentId": 1001, "hostelId": 10, "roomId": 105, "bedNumber": "Bed-A", "joiningDate": "2026-08-01" }
```

- **Response DTO** (`GenericSuccessDto`).
- **Validation Rules:**
 - Target Bed must be in `Available` status.
 - Student must not already have an active allocation.
- **Auth/Role Requirements:** `Hostel Warden`, `Admin`.
- **Related Tables:** `HostelBeds`, `StudentHostelAllocations`, `Students`.
- **Dependencies:** Hostel Rooms & Beds Setup.

2. Record Night Curfew Attendance

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/hostels/attendance`
- **Request DTO** (`CurfewAttendanceDto`):

```
{ "date": "2026-08-03", "hostelId": 10, "floorLevel": "First Floor", "records": [ { "allocationId": 501, "curfewStatus": "Present", "remarks": "" }, { "allocationId": 502, "curfewStatus": "Absent", "remarks": "Informed leave out with parents" } ] }
```

- **Response DTO** (`GenericSuccessDto`).
- **Auth/Role Requirements:** `Hostel Warden`.
- **Related Tables:** `HostelAttendanceLogs`.
- **Dependencies:** Active Student allocations.

Module J: Transport Management

1. Fetch Vehicle Assignments List

- **HTTP Method:** `GET`
- **Endpoint URL:** `/api/transport/vehicle-assignments`
- **Response DTO** (`List`): Returns route, vehicle, driver, and attendant relationships.
- **Auth/Role Requirements:** JWT Authentication. Roles: `Transport Manager`, `Admin`.
- **Related Tables:** `VehicleAssignments`, `Vehicles`, `Drivers`, `Routes`.

2. Create Vehicle Assignment

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/transport/vehicle-assignments`
- **Request DTO** (`CreateVehicleAssianmentDto`):

```
{ "vehicleId": "VEH-01", "routeId": "RTE-05", "driverId": "DRV-10", "attendantId": "STF-25", "morningTripTime": "07:00 AM", "eveningTripTime": "04:30 PM", "effectiveFrom": "2026-08-01" }
```

- **Response DTO** (`VehicleAssignmentDto`).
- **Validation Rules:**
 - Vehicle, driver, and route must not have overlapping active assignments.
- **Auth/Role Requirements:** `Transport Manager`, `Admin`.

Module K: Leaves & Payroll ERP

1. Run Payroll Processing Cycle

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/hr/payroll/runs`
- **Purpose:** Evaluates leaves, LOP, and gross salary to generate draft payslips.
- **Request DTO (`ProcessPayrollRunDto`):**

```
{ "month": "August-2026", "branch": "Main Campus", "calculationMethod": "Working Days" }
```

- **Response DTO (`PayrollRunSummaryDto`):**

```
{ "runId": "RUN-881", "processedCount": 42, "totalEarnings": 1850000, "totalDeductions": 150000, "netPayout": 1700000 }
```

- **Auth/Role Requirements:** `HR`, `Principal`, `Admin`.
- **Related Tables:** `PayrollRuns`, `Payslips`, `StaffLeaveBalances`.
- **Dependencies:** Staff profiles, Leaves record.

2. Disburse Salary Payslips (Release Payment)

- **HTTP Method:** `POST`
- **Endpoint URL:** `/api/hr/payroll/runs/{id}/disburse`
- **Request DTO (`DisbursePayrollDto`):**

```
{ "disbursementDate": "2026-08-05", "paymentAccount": "Salary Account" }
```

- **Response DTO (`GenericSuccessDto`).**
- **Auth/Role Requirements:** `Accountant`, `Principal`, `Admin`.
- **Related Tables:** `PayrollRuns`, `Payslips`, `FinanceTransactions`.

4. Comprehensive Form Integration Mapping

The frontend components contain various inputs that map directly to database columns. Below is the validation mapping to prevent schema conflicts:

Form A: Student Admission Application Form

(Component: StudentFormModal.tsx / AdmissionsView.tsx)

Frontend Fields	Data Type	Database Field	Validation / Constraints
`firstName` / `lastName`	String	`Students.FirstName` / `LastName`	Required, Max 100 characters.
`admissionNo`	String	`Students.AdmissionNo`	Unique Index key, auto-generated index.
`dob`	Date	`Students.Dob`	Required, must check age ranges (>= 3 years).

Frontend Fields	Data Type	Database Field	Validation / Constraints
`email`	String	`Students.Email`	Optional, must match regular expression email schema.
`phone`	String	`Students.Phone`	Max 15 chars, digits only.
`fatherPhone`	String	`Students.FatherPhone`	Required, 10 digits.
`studentType`	Enum	`Students.StudentType`	`Day Scholar` or `Hosteller` only.
`transportRequired`	Boolean	`Students.TransportRequired`	Default `false`.
`hostelRoom`	String	`Students.HostelRoomNumber`	Foreign Key `Rooms.RoomNumber` when Hosteller.

Form B: Staff Profile Detailed Form

(Component: StaffFormModal.tsx)

Frontend Fields	Data Type	Database Field	Validation / Constraints
`empld`	String	`StaffProfiles.Empld`	Unique Index key, format matches `EMP-XXX`.
`employeeCategory`	Enum	`StaffProfiles.EmployeeCategory`	`Teacher` or `Staff`.
`department`	String	`StaffProfiles.DepartmentName`	Matches master `Departments.Name`.
`salary`	Decimal	`StaffProfiles.Salary`	Min value 0.
`bankDetails.accountNumber`	String	`StaffBankDetails.AccountNumber`	Required, digits only, length 9 to 18.
`bankDetails.ifscCode`	String	`StaffBankDetails.IfscCode`	Required, alphanumeric, 11 characters.

5. System Mismatches & Missing Backend Features

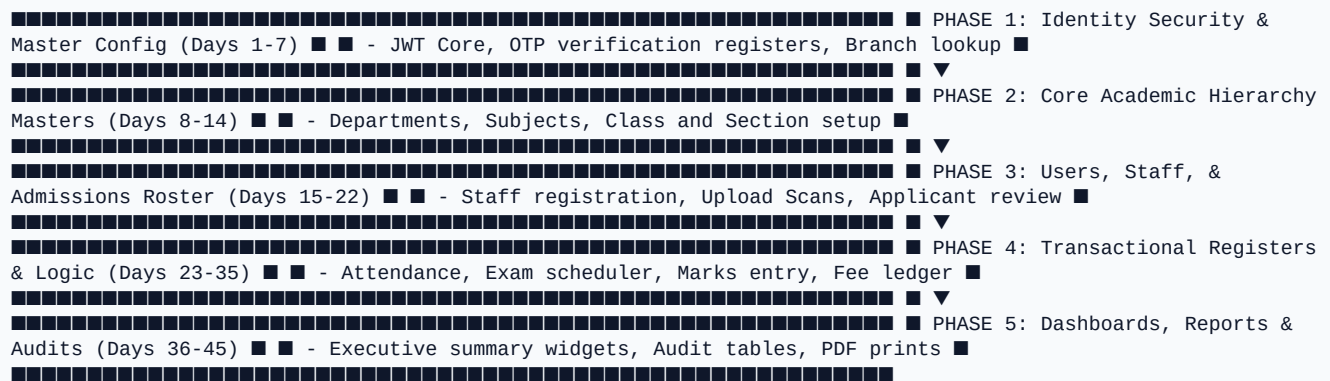
A thorough review of the frontend views reveals the following missing endpoints or data mismatches:

- **Vite Dynamic Submenu Roles Mismatch:**
 - **Issue*:* Comparisons in `Sidebar.tsx` check against `Class Teacher` (e.g. `role === 'Class Teacher'`), but ASP.NET Identity stores role mappings as `Teacher` or `Staff`.
 - **Resolution*:* Add a secondary database flag `IsClassTeacher` to `StaffProfiles` and map it inside the token claims list rather than creating a separate User Role.
- **Missing JWT Token Refresh:**
 - **Issue*:* Frontend lacks logic to request silent token updates, causing session drops during long evaluation entry runs.
 - **Resolution*:* Deploy `/api/auth/token/refresh` exposing secure HttpOnly Refresh Cookies.
- **No Lock Status Endpoint for Marks:**

- *Issue*: Evaluators can modify draft marks inside `MarksEntryView.tsx` forever until published, but there is no middle "Review Lock" step.
- *Resolution*: Expose `POST /api/exams/marks/{id}/lock` to freeze editing during audit checks.

6. Implementation Roadmap & Development Sequence

We will execute the API development cycle in the following sequence to guarantee no endpoint is blocked by missing masters:



Verification & Testing Framework

- **Automated Unit Tests:** Create `xUnit` test suites mapping DTO validation constraints for `LoginRequestDto`, `BulkAttendanceSubmitDto`, and `SubmitMarksRequestDto`.
- **API Route Checks:** Verify payload endpoints against Postman/Swagger JSON collections.
- **Integrity Constraints:** Verify SQL foreign key constraints to prevent orphaned logs in attendance when students exit.

7. Operational Status Checklist

Core Setup APIs

- [] `POST /api/auth/login` (Auth Setup) - ■ Pending
- [] `POST /api/auth/otp/send` (MFA flow) - ■ Pending
- [] `POST /api/auth/otp/verify` (MFA validation) - ■ Pending
- [] `GET /api/lookups/branches` (Branches lists) - ■ Pending
- [] `GET /api/lookups/academic-years` (Active terms) - ■ Pending

Academic Module APIs

- [] `GET /api/departments` (Departments list) - **Pending**
- [] `POST /api/departments` (Create department) - **Pending**
- [] `GET /api/subjects` (Subjects catalog) - **Pending**
- [] `POST /api/subjects` (Add subject) - **Pending**

- [] `GET /api/classes` (Classes setup list) - ■ Pending

Student & Admission APIs

- [] `GET /api/admissions` (Application directory) - ■ Pending
- [] `POST /api/admissions` (New application registry) - ■ Pending
- [] `POST /api/admissions/{id}/enroll` (Enroll applicant) - ■ Pending
- [] `GET /api/students` (Student roster list) - ■ Pending
- [] `POST /api/students` (Create student) - ■ Pending
- [] `POST /api/students/promote` (Students promotion history) - ■ Pending

Attendance & Grading APIs

- [] `GET /api/attendance/students` (Attendance checklist) - ■ Pending
- [] `POST /api/attendance` (Bulk attendance registry) - ■ Pending
- [] `POST /api/exams/marks` (Submit exam grades) - ■ Pending
- [] `PUT /api/exams/marks/{id}/revalue` (Revaluation audit) - ■ Pending

Finance & Admin Operations

- [] `POST /api/finance/fee-structures` (Tuition setups) - ■ Pending
- [] `POST /api/finance/payments` (Collect tuition fee) - ■ Pending
- [] `POST /api/hr/payroll/runs` (Calculate gross payouts) - ■ Pending
- [] `POST /api/hr/payroll/runs/{id}/disburse` (Disburse payroll payslip) - ■ Pending

Status Mapping: ■ Pending (Needs backend execution), ■ Completed (Ready), ■ Dependency Blocked.